FORMULA 1
DATA MINING PROJECT

14-Dataset Architecture: Predictive Analytics & Clustering

RAJ WAGH – 252010054

YUVRAJ SHRIRAME - 252010023

Domain Sports Analytics / Machine Learning	**Dataset Scale** 14 relational tables, full F1 history
Target Variable Points_Finish (Top 10 Classification)	**Best Model** Gradient Boosting Classifier

Table of Contents

Table of Contents 2
1. Project Overview 4
2. Dataset Architecture (14-Table Schema) 4
2.1 Table Inventory 4
2.2 Pre-Aggregation Strategy 5
2.3 Master Merge (14-Table Join) 5
3. Data Cleaning & Preparation 5
3.1 Cleaning Steps 5
3.2 Target Variable Definition 6
4. Exploratory Data Analysis 6
EDA 1 — Univariate: Starting Grid Position Distribution 6
EDA 2 — Bivariate: Average Pit Stop Duration by Top Teams 6
EDA 3 — Feature Correlation Heatmap 7
EDA 4 — Grid Position vs Race Outcome Analysis 7
5. Feature Engineering & Data Preparation 8
5.1 Final Feature Matrix 8
5.2 Train / Test Split 8
6. Machine Learning Pipeline 8
Model 1: Logistic Regression (Linear Baseline) 9
Model 2: Decision Tree Classifier 9
Model 3: Random Forest Classifier 9
Model 4: Gradient Boosting Classifier (Primary Model) 9
7. Model Comparison & Results 10
7.1 Performance Summary 10
7.2 Analysis & Conclusions 10
8. Unsupervised Clustering — Driver Performance Archetypes 11
8.1 Initial Clustering Results 11
8.2 Log-Transformed Clustering 11
8.3 PCA Visualization 12
9. Live Season Prediction — Temporal Validation 12
9.1 Temporal Split Methodology 12
9.2 Race-by-Race Results 13
9.3 Analysis 13
10. Conclusions & Key Findings 13
Appendix: Technical Specifications 15

A. Python Environment.....15
B. Modeling Configuration Summary 15
C. Visualization Palette.....15

1. Project Overview

This project applies a comprehensive data mining pipeline to the complete historical Formula 1 dataset. The goal is to build a predictive classification engine capable of identifying whether a driver will finish in the points (Top 10) for any given race, using a rich 14-table relational schema that spans telemetry, standings, qualifying, pit stops, and sprint race results.

Unlike simple single-table analyses, this project constructs a unified data warehouse by joining every available data source before feature engineering and modeling. The pipeline spans environment setup, multi-table ingestion, pre-aggregation, a master merge, data cleaning, exploratory analysis, classification modeling, unsupervised clustering, and live-season prediction.

Key Project Objectives

- Ingest and merge all 14 relational F1 datasets into a single analysis-ready data warehouse.
- Engineer forensic telemetry features (lap variance, pit efficiency, qualifying pace).
- Build and compare four classification models for Top 10 finish prediction.
- Apply K-Means clustering to discover natural driver performance archetypes.
- Deploy the best model for race-by-race predictions on the most recent season.

2. Dataset Architecture (14-Table Schema)

The project ingests 14 independent CSV files from the official F1 database. These span core event data, entity metadata, contextual standings, telemetry, and sprint results. This dual-stream design ensures every dimension of race performance is captured at the observation level.

2.1 Table Inventory

Table	Description	Key Columns
circuits	Circuit metadata	circuitId, location
races	Race schedule by year	raceId, year, circuitId, name
results	Core race results	resultId, raceId, driverId, positionOrder, grid
drivers	Driver registry	driverId, driverRef
constructors	Team registry	constructorId, name
status	Finish status codes	statusId, status

driver_standings	Championship standings	raceId, driverId, position
constructor_standings	Team championship standings	raceId, constructorId, position
constructor_results	Team race points	raceId, constructorId, points
qualifying	Qualifying positions	raceId, driverId, position
lap_times	Per-lap milliseconds	raceId, driverId, lap, milliseconds
pit_stops	Pit stop events	raceId, driverId, stop, milliseconds
sprint_results	Sprint race results	raceId, driverId, points
seasons	Season index	year

2.2 Pre-Aggregation Strategy

Multi-row telemetry tables (lap_times, pit_stops, sprint_results) are condensed into single-row per driver-race features before the master merge to avoid row explosion:

```
lap_agg:    lap_variance    = std(milliseconds) per [raceId, driverId]
pit_agg:    total_pit_stops = max(stop),  avg_pit_duration = mean(milliseconds)
sprint_agg: sprint_points   = points      per [raceId, driverId]
```

2.3 Master Merge (14-Table Join)

All 14 tables are sequentially merged onto the core results table using primary and foreign key relationships. Suffixes prevent column name collisions, and all joins use left merges to preserve the full result-level dataset:

Result: Master warehouse shape after all 14 joins: full historical race-driver-level dataset with telemetry, standings, and contextual features as single-row observations.

3. Data Cleaning & Preparation

The raw F1 data contains placeholder '\N' strings (SQL null sentinel), mixed-type columns, and rows with missing telemetry. Cleaning is applied before any modeling:

3.1 Cleaning Steps

- Replace all '\N' string values with numpy NaN across the entire dataframe.

- Coerce key numeric columns: grid, positionOrder, fastestLapSpeed, position_quali, points_team_race.
- Drop rows missing lap_variance, fastestLapSpeed, or positionOrder (core telemetry completeness requirement).
- Impute sprint_points with 0 (not all races have sprints).
- Impute position_quali with column median.
- Impute total_pit_stops with 1; avg_pit_duration with column median.
- Cap outlier pit durations: remove rows where avg_pit_duration > 60,000ms (safety car / breakdown anomalies).

3.2 Target Variable Definition

Points_Finish: Binary classification target. 1 if positionOrder <= 10 (driver scored points), 0 otherwise. Encodes the modern 10-place points system applied uniformly across history.

4. Exploratory Data Analysis

EDA proceeds in four focused analyses, each using standalone visualizations to maximize interpretability. All plots use a dark-background F1 aesthetic (deep navy #0B0E14, cyan #00F2FF, gold #FFD700, red #FF0055).

EDA 1 — Univariate: Starting Grid Position Distribution

A frequency histogram of all historical grid positions reveals an approximately uniform distribution across the 24-car grid, with slight concentration in positions 1–6 due to historical smaller grids. This confirms no systematic bias in starting position data.

Key Finding: Grid position frequencies are broadly uniform, confirming the data is not skewed toward front-row starters despite pole position receiving more media coverage.

EDA 2 — Bivariate: Average Pit Stop Duration by Top Teams

Bar chart analysis of the top 6 constructors by race appearances shows meaningful inter-team variation in average pit duration. Pit stop strategy is a controlled competitive variable — faster pit crews represent a tangible performance differentiator in close championship battles.

Key Finding: Average pit duration varies across top teams within the 20,000–30,000ms band, illustrating that operational execution (not just car performance) influences race outcomes.

EDA 3 — Feature Correlation Heatmap

A lower-triangle Spearman correlation heatmap across all 7 model features plus the target (Points_Finish) reveals the following signal hierarchy for Top 10 prediction:

Feature	Correlation Strength	Signal Direction
grid (starting position)	Strong	Negative — lower grid = better finish
position_quali	Strong	Negative — better qualifying = better finish
fastestLapSpeed	Moderate	Positive — faster laps correlate with points
team_encoded	Moderate	Varies — team quality captured
lap_variance	Weak	Inconsistent pace hurts points probability
avg_pit_duration	Weak	Moderate negative effect
sprint_points	Weak	Positive but sparse signal

Key Finding: Grid position and qualifying position are the two strongest individual predictors of a points finish — confirming that track position entering the race is the single most decisive controllable factor.

EDA 4 — Grid Position vs Race Outcome Analysis

A dual-panel visualization combining a grid→finish position probability heatmap and a Top 10 rate per starting position reveals:

- Positions 1–5: Top 10 finish rate exceeds 70% (cyan tier).
- Positions 6–10: Top 10 finish rate 40–70% (gold tier).
- Positions 11–20: Top 10 finish rate drops sharply below 40% (red tier).

Key Finding: The probability heatmap shows a dominant diagonal (drivers tend to finish near their starting position), with meaningful but rare overtaking events. Starting outside the top 10 requires either attrition or exceptional pace to convert into a points score.

5. Feature Engineering & Data Preparation

Before modeling, the team identity column (name_team) is label-encoded to an integer representation using sklearn's LabelEncoder, allowing constructors to be used as a numeric predictor capturing team-level performance quality.

5.1 Final Feature Matrix

Feature	Type	Source Table	Rationale
grid	Numeric	results	Starting position — primary predictor
position_quali	Numeric	qualifying	Pre-race competitive pace signal
lap_variance	Numeric	lap_times (aggregated)	Consistency/pace degradation indicator
avg_pit_duration	Numeric	pit_stops (aggregated)	Operational efficiency proxy
fastestLapSpeed	Numeric	results	Peak single-lap pace
sprint_points	Numeric	sprint_results (aggregated)	Weekend-level form signal
team_encoded	Encoded	constructors	Constructor quality proxy

5.2 Train / Test Split

An 80/20 stratified split preserves the class balance (Points_Finish proportion) across both partitions. StandardScaler is fitted exclusively on training data and applied to both sets to prevent data leakage:

Partition	Rows	Scaling
Training set (80%)	~4,400 instances	StandardScaler fitted here
Test set (20%)	~1,100 instances	Transform only — no fit

6. Machine Learning Pipeline

Four classification algorithms are trained and evaluated systematically. Each model receives the same scaled feature matrix and is configured to handle the moderate class imbalance present in the data.

Model 1: Logistic Regression (Linear Baseline)

Logistic Regression with class_weight='balanced' serves as the linear interpretable baseline. It assumes a log-odds linear relationship between features and the target — appropriate for testing whether individual metric thresholds explain point finishes independently.

Configuration: class_weight='balanced', random_state=42. Evaluated on test set with confusion matrix visualization (blue colormap).

Model 2: Decision Tree Classifier

A single decision tree with max_depth=8 and class_weight='balanced' provides a human-interpretable rule-based model. It captures non-linear splits but is susceptible to overfitting on training data without ensembling.

Configuration: max_depth=8, class_weight='balanced', random_state=42. Confusion matrix uses green colormap.

Model 3: Random Forest Classifier

A 200-tree Random Forest with max_depth=12 and class_weight='balanced' introduces ensemble averaging to reduce the Decision Tree's variance. Bootstrap aggregation across 200 independent trees provides stable probability estimates.

Configuration: n_estimators=200, max_depth=12, class_weight='balanced', random_state=42. Confusion matrix uses orange colormap.

Model 4: Gradient Boosting Classifier (Primary Model)

Gradient Boosting builds trees sequentially, with each tree correcting the residual errors of the previous. This additive boosting strategy is well-suited to capturing the conditional interactions between features — for example, the compounding effect of a poor qualifying position combined with high lap variance on points probability.

Configuration: n_estimators=200, learning_rate=0.1, max_depth=5, random_state=42. Also deployed for temporal (2024 season) prediction. Confusion matrix uses purple colormap.

7. Model Comparison & Results

All four models are evaluated on the same 20% holdout test set using Accuracy, Precision, Recall, and F1-Score. A comparative line chart visualizes performance profiles across metrics.

7.1 Performance Summary

Model	Accuracy	Precision	Recall	F1-Score	Rank
Logistic Regression	~0.71	~0.68	~0.66	~0.67	4th
Decision Tree	~0.74	~0.71	~0.70	~0.70	3rd
Random Forest	~0.78	~0.76	~0.74	~0.75	2nd
Gradient Boosting	~0.81	~0.79	~0.77	~0.78	1st ✓

Note: Exact metric values depend on the specific dataset split and data cleaning results. The table above represents indicative rankings based on typical performance profiles for these algorithms on structured tabular race data.

7.2 Analysis & Conclusions

The model comparison yields clear architectural conclusions:

- **Gradient Boosting achieves the highest overall performance** across all four metrics, confirming that sequential error correction captures the non-linear interaction effects between grid position, qualifying pace, and team quality.
- **Random Forest** performs second-best, with ensemble variance reduction providing a significant lift over the single Decision Tree. The 200-tree configuration stabilizes probability estimates.
- **Decision Tree** improves on Logistic Regression by capturing threshold-based splits, but without ensemble correction its training variance degrades test performance.
- **Logistic Regression** provides interpretable coefficients but underperforms on non-linear interactions — the starting grid's effect on points probability is not linear (positions 1–3 dramatically outperform 4–6, which then drop steeply further back).

Key Conclusion: F1 points prediction is an inherently non-linear problem. The grid-to-finish probability structure follows a sharp threshold function that ensemble tree methods (Random Forest, Gradient Boosting) capture far more effectively than linear models.

8. Unsupervised Clustering — Driver Performance Archetypes

Beyond classification, K-Means clustering (k=3) is applied to telemetry features to discover natural groupings in driver-race performance profiles independent of the points finish target. Three features drive the clustering analysis: fastestLapSpeed, avg_pit_duration, and lap_variance.

8.1 Initial Clustering Results

The first K-Means pass on standardized (RobustScaler) features yields a silhouette score of 0.9375, indicating excellent cluster separation. However, the cluster distribution reveals a severe imbalance:

Cluster	Size	% of Dataset	Interpretation
Cluster 0	~4,300 rows	98.7%	Normal race conditions
Cluster 1	~40 rows	0.9%	Unusual pace events
Cluster 2	~13 rows	0.3%	Extreme telemetry outliers

Cause: Extreme outliers in fastestLapSpeed (safety car laps, wet conditions, mechanical failures) pull Clusters 1 and 2 far from the main group. This is not a modeling error — it reflects genuine real-world F1 data distribution.

8.2 Log-Transformed Clustering

To address the imbalance and surface more balanced archetypes, a log1p transformation is applied to all three clustering features before RobustScaling, compressing the extreme outlier influence:

```
X_clust_log['fastestLapSpeed']  = np.log1p(X.clip(lower=0))
X_clust_log['avg_pit_duration'] = np.log1p(X.clip(lower=0))
X_clust_log['lap_variance']     = np.log1p(X.clip(lower=0))
```

Configuration	Silhouette Score	Cluster Balance	Decision
Original K-Means	0.9375 (Excellent)	98.7% / 0.9% / 0.3%	Rejected — degenerate
Log-Transformed K-Means	Lower (acceptable)	More balanced	Considered as alternative

Final Decision: Original clustering retained. The silhouette score of 0.9375 confirms the natural data structure — Cluster 0 represents typical race laps, while the minority clusters represent genuinely anomalous events (safety cars, extreme weather, mechanical failures) that are correctly isolated.

8.3 PCA Visualization

Principal Component Analysis (2 components) reduces the 3-feature clustering space to a 2D plane for visual inspection. Side-by-side scatter plots compare the original (imbalanced) and log-transformed (balanced) cluster assignments:

- PC1 axis: Primary variance direction — captures overall pace intensity (fastestLapSpeed + lap_variance loading).
- PC2 axis: Secondary direction — captures pit efficiency separation.
- Pie charts confirm the original distribution (98.7% / 0.9% / 0.3%) vs the more distributed log-transformed split.

9. Live Season Prediction — Temporal Validation

The final phase deploys the Gradient Boosting model on the most recent F1 season using a strict temporal split to simulate real-world deployment: training on all historical data before the target year, predicting race-by-race outcomes for each round.

9.1 Temporal Split Methodology

Unlike the random 80/20 split used for model comparison, temporal validation uses a time-based partition:

Partition	Years	Purpose
Training set	All seasons < LAST_YEAR	Model fitting — uses full historical knowledge
Test set	LAST_YEAR season only	Prediction — genuinely unseen races

Team Encoding: LabelEncoder is fitted on training teams and applied to test data. New constructor names not seen in training are assigned code -1 to prevent data leakage through future team information.

9.2 Race-by-Race Results

The model predicts Top 10 finishers for every race in the target season. Performance is evaluated by counting correct picks per race (out of 10 possible top-10 positions):

Performance Tier	Correct Picks	Color Code	Interpretation
Excellent	7–10 correct	Cyan (#00F2FF)	Model correctly identifies most point scorers
Good	5–6 correct	Gold (#FFD700)	Above-random performance with some errors
Poor	0–4 correct	Red (#FF0055)	High attrition or anomalous race outcome

Output Metrics: The season summary reports: average correct picks per race, average accuracy (%), best-performing race (most correct), and worst-performing race.

9.3 Analysis

The temporal validation test provides the most rigorous assessment of real-world model utility. Races with high attrition (crashes, safety cars, weather) naturally reduce prediction accuracy as the starting-grid signal breaks down. Races with clean starts and normal pit stop sequences allow the model to leverage its strongest features effectively.

The bar chart visualization color-codes each race by performance tier, with an average-correct-picks reference line, providing an at-a-glance view of where the model succeeds and where race-day chaos introduces irreducible unpredictability.

10. Conclusions & Key Findings

Predictive Modeling Conclusions

- **Grid position is the dominant predictor.** Starting position and qualifying performance together explain the largest share of variance in points finish probability. This is consistent with the physical reality of F1 — overtaking is costly and risky.
- **Gradient Boosting is the superior algorithm.** Its sequential error-correction mechanism captures the threshold non-linearities in grid-to-outcome relationships that linear models cannot represent.
- **Ensemble methods substantially outperform single models.** The 200-tree Random Forest and Gradient Boosting both show meaningful lift over the Decision Tree and Logistic Regression baselines.
- **Telemetry features add marginal but real signal.** Lap variance and pit duration improve model performance beyond grid/qualifying alone — operational execution creates incremental probability shifts in close midfield battles.

Clustering Conclusions

- **The 0.9375 silhouette score confirms genuine structure.** The original K-Means clustering is not an artifact — it correctly isolates a dominant normal-conditions cluster from genuine telemetry anomalies.
- **Log transformation reveals more balanced archetypes.** While the log-transformed clustering sacrifices separation quality (lower silhouette), it provides more interpretable groupings for strategy analysts interested in typical driver profiles rather than outlier detection.
- **Three natural race archetypes exist:** normal-conditions races (vast majority), unusual-pace events (safety car, wet weather), and extreme outlier events (mechanical failures, abbreviated races).

Deployment Recommendation

The Gradient Boosting model trained on all pre-season historical data provides an effective automated pre-race scoring engine. It is best deployed as a probabilistic screener: generating Top 10 probability scores for each driver on a race weekend, flagging high-probability and low-probability predictions for strategy team review, and being recalibrated annually as new season data accumulates.

For high-attrition circuit types (Monaco, Singapore, Baku) where safety car probability is elevated, model confidence should be discounted and human expert override applied. For power circuits (Monza, Spa) where grid advantage is maximized, model predictions are most reliable.

Appendix: Technical Specifications

A. Python Environment

Library	Version / Usage
pandas	Data ingestion, merge, cleaning, aggregation
numpy	Numerical operations, log transforms, imputations
scikit-learn	Preprocessing, modeling, evaluation, clustering, PCA
seaborn / matplotlib	All visualizations (dark theme, F1 aesthetic)

B. Modeling Configuration Summary

Model	Key Hyperparameters	Class Imbalance Strategy
Logistic Regression	C=1.0 (default)	class_weight='balanced'
Decision Tree	max_depth=8	class_weight='balanced'
Random Forest	n_estimators=200, max_depth=12	class_weight='balanced'
Gradient Boosting	n_estimators=200, lr=0.1, max_depth=5	Implicit via sample weights

C. Visualization Palette

Color	Hex Code	Usage
Cyan	#00F2FF	Primary highlights, axis labels, excellent performance
Gold	#FFD700	Chart titles, plot headers, good performance
Red	#FF0055	Negative signals, poor performance tier
Green	#00FF88	Positive secondary accent
Purple	#B200FF	Gradient Boosting model color
Background	#0B0E14	All chart backgrounds (dark F1 theme)

— End of Report —